

UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA E MATEMATICA APPLICATA



CORSO DI LAUREA TRIENNALE IN
INGEGNERIA DELL'INFORMAZIONE PER LA MEDICINA DIGITALE
INGEGNERIA INFORMATICA

Tirocinio in: ACADEMY MEDICAL WEARABLE DEVICES

Alvea

Sistema IoT per il monitoraggio dell'asma pediatrico
Report Tecnico del Progetto

Docenti:

Alessia Saggese
Martino Giaquinto

Gruppo 04:

Domenico Chiacchio
Antonio Crisci
Rita Pia Iacoviello
Silvia Liguoro

Repository GitHub: <https://github.com/acrisci05/Alvea>

ANNO ACCADEMICO 2025/2026

Abstract

Il presente documento espone la progettazione e l'implementazione di Alvea, un sistema IoT (Internet of Things) per il monitoraggio continuo di pazienti pediatrici affetti da asma, sviluppato dal Gruppo 04 nell'ambito dell'Academy Medical Wearable Devices.

Alvea è un dispositivo indossabile da caviglia che, da un unico sensore analogico ECG, ricava il battito cardiaco e — tramite la tecnica EDR (ECG-Derived Respiration) — la frequenza respiratoria, parametro chiave per il riconoscimento precoce di una crisi asmatica. Misura, inoltre, la temperatura cutanea con un termistore NTC di precisione e il livello di batteria attraverso un partitore di resistenze, oltre a rilevare l'aderenza degli elettrodi (leads-off).

Il sistema realizza una catena completa “dal sensore al medico”: il dispositivo pubblica la telemetria via MQTT; un backend FastAPI la valida, la storicizza e la ritrasmette in tempo reale all'app mobile (React Native) via WebSocket; un percorso parallelo (Node-RED → InfluxDB → Grafana) alimenta la dashboard clinica del medico.

L'accesso è protetto da autenticazione JWT con ruoli distinti (caregiver e medico), isolamento dei dati per proprietà e audit log; l'infrastruttura server è interamente orchestrata con Docker Compose.

Il documento, dopo un'analisi contestualizzata dei requisiti funzionali e non funzionali, descrive l'architettura, i singoli componenti, i protocolli di comunicazione e la sicurezza, concludendo con la validazione e una valutazione critica dei risultati.

Indice

Abstract	2
Indice	3
1. Introduzione.....	5
1.1 Contesto e obiettivo del progetto.....	5
1.2 Natura didattica del progetto	5
1.3 Architettura del sistema.....	5
1.4 Organizzazione del documento	6
2. Analisi dei requisiti.....	6
2.1 Requisiti funzionali.....	6
2.2 Requisiti non funzionali	7
2.3 Casi d'uso principali.....	8
2.4 Stato di implementazione.....	9
3. Ambiente e strumenti.....	9
3.1 Stack tecnologico	9
3.2 Architettura a container e porte	9
3.3 Struttura del repository	10
3.4 Viste architetturali (modello 4+1).....	10
4. Dispositivo indossabile (firmware).....	10
4.1 Sensoristica.....	10
4.2 Elaborazione a bordo	11
4.3 Comunicazione e allarmi	11
5. Backend e gestione dei dati.....	12
5.1 API REST e autenticazione	12
5.2 Ingest e canali in tempo reale	12
5.3 Persistenza: database relazionale e InfluxDB	12
5.4 Soglie cliniche e gestione degli alert	13
5.5 Scheda paziente e auto-assegnazione	13
6. Modello dei dati	13
7. Protocolli di comunicazione.....	15
7.1 Firmware ↔ Backend	15
7.2 App ↔ Backend.....	15
7.3 Configurazione da remoto	15
8. Applicazione mobile	16
8.1 Architettura a livelli	16
8.2 Schermata di monitoraggio e resilienza.....	16
8.3 Notifiche e sicurezza della sessione	16
9. Dashboard medico (Grafana).....	17
10. Sicurezza e privacy.....	17
10.1 Autenticazione e ruoli.....	17

10.2 Isolamento dei dati e audit log.....	17
10.3 Gestione dei segreti e avvertenze.....	17
11. Scelte progettuali di sintesi	18
12. Test e validazione.....	18
13. Conclusioni.....	18
13.1 Obiettivi raggiunti.....	18
13.2 Valutazione del risultato e limiti	18
13.3 Sviluppi futuri.....	19
Appendice A — Glossario e acronimi.....	20

1. Introduzione

1.1 Contesto e obiettivo del progetto

L'asma è una delle patologie croniche più diffuse in età pediatrica e le sue crisi possono evolvere rapidamente. Nel bambino la comunicazione dei sintomi è spesso difficile o tardiva: un episodio può aggravarsi prima che il genitore se ne accorga. Un monitoraggio continuo e non invasivo dei parametri respiratori e cardiaci consente di cogliere precocemente i segni di un peggioramento, in particolare l'aumento della frequenza respiratoria (tachipnea), che tipicamente precede la fase acuta.

Alvea nasce con l'obiettivo di realizzare un dispositivo indossabile leggero — una cavigliera adatta a un bambino — e l'infrastruttura software che lo accompagna, in grado di acquisire i parametri vitali in modo continuo, renderli disponibili in tempo reale al genitore tramite un'applicazione mobile, fornire al medico una vista clinica storica e segnalare automaticamente, attraverso notifiche, le condizioni di rischio. Il sistema è pensato per un contesto domiciliare, dove semplicità d'uso e affidabilità della segnalazione sono prioritarie.

1.2 Natura didattica del progetto

È importante inquadrare correttamente l'ambito: *Alvea* è un progetto didattico e dimostrativo, non un dispositivo medico certificato. Non sostituisce la sorveglianza di un adulto né strumenti clinici approvati, e i valori, le soglie e gli allarmi hanno finalità di laboratorio. Coerentemente, il dispositivo è alimentato solo a bassa tensione (cella LiPo o power bank) e tutto il traffico clinico resta sulla rete locale, senza invio a servizi cloud di terze parti.

1.3 Architettura del sistema

Alvea adotta un'architettura IoT a livelli, fondata sul paradigma publish/subscribe. Il dispositivo non comunica mai direttamente con l'app o con il database: pubblica i propri dati su un broker MQTT (Mosquitto), dal quale i consumatori a valle li prelevano. Questa separazione fa sì che ogni componente conosca soltanto il “contratto” con i propri vicini, semplificando sviluppo, test e sostituzione di un componente. Il sistema si articola in tre percorsi paralleli che condividono lo stesso payload:

- **Percorso clinico:** Dispositivo → MQTT → Node-RED → InfluxDB → Grafana, che alimenta i grafici storici per il medico.
- **Percorso applicativo:** Dispositivo → MQTT → Backend → database relazionale → WebSocket → App, che fornisce al genitore dati live e allarmi.
- **Percorso di controllo (ritorno):** Backend → MQTT → Dispositivo, per i comandi di configurazione (frequenza di invio, associazione paziente).

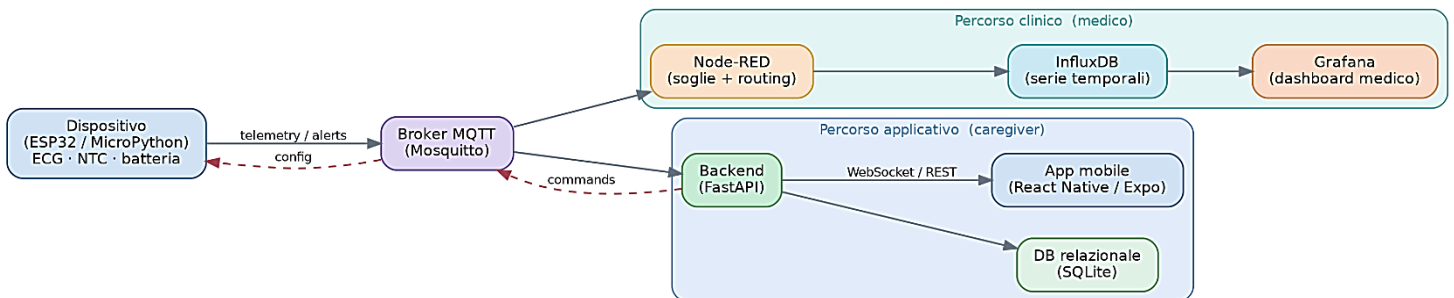


Figura 1 — Architettura a livelli di *Alvea*: i tre percorsi paralleli e il canale di controllo di ritorno (tratteggiato).

La tabella seguente riassume i componenti principali e il loro ruolo.

Componente	Tecnologia	Ruolo nel sistema
Dispositivo indossabile	ESP32 / MicroPython	Acquisisce i segnali, elabora a bordo, pubblica via MQTT
Broker di messaggi	Mosquitto (MQTT)	Smista telemetria, allarmi e comandi
Backend	FastAPI (Python)	API, autenticazione, ingest, soglie, realtime
Motore regole	Node-RED	Instradamento verso InfluxDB e ripubblicazione degli alert
Database storico	InfluxDB	Serie temporali per la dashboard clinica
Database applicativo	SQLite	Account, device, letture, allarmi, schede paziente, soglie, audit
App mobile	React Native / Expo	Monitoraggio per il genitore/caregiver
Dashboard	Grafana	Vista clinica storica per il medico

Tabella 1 — Componenti principali del sistema Alvea.

1.4 Organizzazione del documento

Il documento è strutturato come segue. La Sezione 2 presenta l'analisi contestualizzata dei requisiti e i casi d'uso; la Sezione 3 descrive l'ambiente e gli strumenti. Le Sezioni 4 e 5 approfondiscono il dispositivo e il backend, la Sezione 6 il modello dei dati e la Sezione 7 i protocolli di comunicazione. Le Sezioni 8 e 9 riguardano l'app mobile e la dashboard del medico, la Sezione 10 la sicurezza, la Sezione 11 riepiloga le scelte progettuali, la Sezione 12 la validazione e la Sezione 13 trae le conclusioni; un glossario chiude il documento.

2. Analisi dei requisiti

I requisiti sono stati formalizzati con codici RQ-XX e distinti in funzionali (F) e non funzionali (NF), con una priorità (0 = massima) e uno stato di implementazione. Di seguito sono presentati in forma contestualizzata rispetto allo scenario d'uso, il monitoraggio domiciliare di un bambino asmatico, e poi riepilogati in forma tabellare.

2.1 Requisiti funzionali

Sul versante del rilevamento, la scelta progettuale più caratterizzante è l'uso di un unico sensore biomedicale, l'ECG, dal quale derivare due parametri: il battito cardiaco e la frequenza respiratoria (RQ-01). In un dispositivo indossato da un bambino, ridurre il numero di sensori significa minore ingombro, maggiore comfort e meno punti di guasto; la frequenza respiratoria, ottenuta via EDR, è inoltre il segnale clinicamente più specifico per l'asma. A corredo, un termistore misura la temperatura cutanea (RQ-02) e i pin di leads-off dell'ECG rilevano se il dispositivo è effettivamente aderente alla pelle (RQ-03), informazione essenziale per non confondere un sensore staccato con un'anomalia fisiologica.

Sul versante della comunicazione, il dispositivo pubblica la telemetria a cadenza regolare su un topic MQTT dedicato (RQ-04), con un payload JSON ben definito e condiviso da tutti i componenti (RQ-05); è prevista, a livello di firmware, anche una trasmissione BLE alternativa verso un eventuale ricevitore locale (RQ-06). Il canale è bidirezionale: il dispositivo riceve comandi di configurazione, ad esempio la frequenza di invio o l'associazione a un paziente (RQ-07, RQ-14).

L'elaborazione clinica valuta i parametri rispetto a soglie e genera gli allarmi rilevanti per l'asma, in primo luogo la tachipnea (RQ-08); una regola di buon senso sopprime gli allarmi fisiologici quando il sensore non è a contatto, segnalando in quel caso solo un avviso tecnico (RQ-15). I dati vengono storicizzati su due piani complementari: le serie temporali cliniche su InfluxDB (RQ-09) e le schede paziente (anagrafica, patologie, farmaci, allergie) su un database relazionale (RQ-10).

Infine, la fruizione: il medico dispone di una dashboard Grafana con metriche e allarmi (RQ-11), mentre il genitore usa l'app mobile per il monitoraggio in tempo reale e la ricezione degli alert via WebSocket (RQ-13). Tutti gli accessi ai dati sono protetti da autenticazione JWT con isolamento per account e ruolo (RQ-12).

ID	Descrizione (sintesi)	Pr.	Stato
RQ-01	BPM + frequenza respiratoria (EDR) da un solo ECG	0	Implementato
RQ-02	Temperatura cutanea (termistore NTC)	1	Implementato
RQ-03	Rilevamento aderenza (leads-off)	1	Implementato
RQ-04	Telemetria periodica su MQTT	0	Implementato
RQ-05	Payload JSON standard condiviso	0	Implementato
RQ-06	Trasmissione BLE alternativa (solo firmware)	3	Parziale
RQ-07	Comandi di configurazione via MQTT	0	Implementato
RQ-08	Soglie cliniche e alert (tachipnea)	0	Implementato
RQ-09	Storico serie temporali su InfluxDB	1	Implementato
RQ-10	Schede paziente su DB relazionale	2	Implementato
RQ-11	Dashboard Grafana con metriche e alert	1	Implementato
RQ-12	Autenticazione JWT e isolamento (RBAC)	0	Implementato
RQ-13	Monitor e alert realtime sull'app	1	Implementato
RQ-14	Associazione device-paziente	2	Implementato
RQ-15	Soppressione falsi allarmi fisiologici	1	Implementato

Tabella 2 — Requisiti funzionali (sintesi). RQ-06 è realizzato a livello di firmware ma privo di consumatore lato app (cfr. §2.4).

2.2 Requisiti non funzionali

Sul piano delle prestazioni, l'acquisizione del segnale ECG ad alta frequenza non deve mai bloccare il resto del firmware (RQ-16), e la latenza dall'acquisizione alla visualizzazione deve restare nell'ordine di un paio di secondi (RQ-17); sul piano dell'affidabilità, il dispositivo deve riconnettersi automaticamente a Wi-Fi e broker in caso di caduta, tramite una macchina a stati cooperativa integrata nel ciclo principale (RQ-18).

La sicurezza e la privacy sono trattate come requisiti di prima classe: protezione delle API, hashing delle password, isolamento dei dati per utente (RQ-19) e gestione delle credenziali al di fuori del repository, tramite file esclusi dal versionamento (RQ-20). La portabilità è garantita da uno stack riproducibile con un solo comando (Docker Compose, RQ-21), la sicurezza d'uso dalla scelta di alimentazione a bassa tensione e dalla natura didattica del prototipo (RQ-22), e la manutenibilità dal fatto che simulatore e hardware reale condividono lo stesso schema di payload (RQ-23), rendendo le due modalità intercambiabili.

ID	Descrizione (sintesi)	Categoria	Stato
RQ-16	Esecuzione non bloccante (ciclo cooperativo)	Performance	Implementato
RQ-17	Latenza end-to-end contenuta (~2 s)	Performance	Implementato
RQ-18	Riconnessione automatica Wi-Fi/MQTT	Affidabilità	Implementato
RQ-19	Sicurezza API, hashing, isolamento dati	Sicurezza	Implementato
RQ-20	Credenziali fuori dal repository	Sicurezza	Implementato
RQ-21	Stack riproducibile (Docker Compose)	Portabilità	Implementato
RQ-22	Dispositivo didattico, bassa tensione	Safety	Implementato
RQ-23	Simulatore e HW reale: stesso payload	Manutenibilità	Implementato

Tabella 3 — Requisiti non funzionali (sintesi).

2.3 Casi d'uso principali

Gli attori del sistema sono il Paziente/Caregiver (attore primario, che monitora e consulta), il Medico (attore primario con permessi estesi: configurazione delle soglie, consultazione di tutti i pazienti e dell'audit log) e il Dispositivo ESP32 (attore secondario, che immette telemetria e riceve configurazioni).

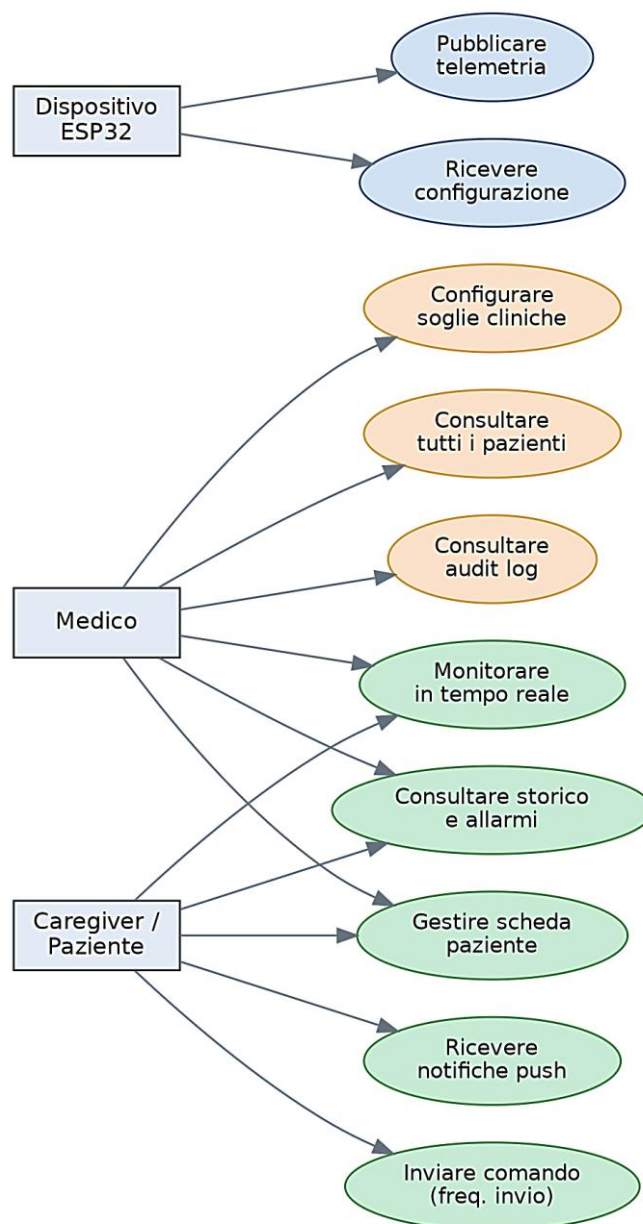


Figura 2 — Diagramma dei casi d'uso (UML). Il medico dispone di tutti i permessi del caregiver più configurazione soglie, vista su tutti i pazienti e audit log.

Il caso d'uso centrale è la visualizzazione in tempo reale.

Precondizioni: l'utente è autenticato e il dispositivo è associato al paziente.

1. Il caregiver apre la schermata di monitoraggio dell'app.
2. L'app apre il canale WebSocket verso il backend (con fallback su polling REST).
3. Il dispositivo pubblica una lettura completa (BPM, respiro, temperatura, batteria).
4. Il backend la riceve, la valida, la storicizza e la inoltra istantaneamente.
5. L'app aggiorna l'interfaccia con i nuovi valori.

Flusso alternativo: se il sensore non è a contatto, il sistema mostra un avviso tecnico e sospende la valutazione fisiologica, evitando falsi allarmi. La lettura resta comunque persistita e visibile anche nella dashboard del medico.

2.4 Stato di implementazione

Tutti i requisiti a priorità massima risultano implementati nel codice sorgente. Alcune funzionalità a priorità più bassa sono presenti in forma parziale, e vengono qui dichiarate esplicitamente per trasparenza:

- **Trasmissione BLE (RQ-06):** è realizzata a livello di firmware (moduli `transport_ble.py`, `main_real_ble.py`, `main_sim_ble.py`) ma non ha, al momento, un consumatore nell'app mobile, che comunica esclusivamente via REST/WebSocket.
- **Distinzione dei ruoli nell'app:** la logica RBAC completa (ruolo, controllo di proprietà, permessi, audit) è pienamente implementata lato backend. Nell'app mobile, invece, la visibilità degli strumenti riservati al medico è governata da un flag statico di configurazione (`SHOW_MEDICO_TOOLS`): l'aggancio al ruolo effettivo restituito dal login resta un affinamento previsto ma non ancora attivo in ogni schermata.
- **Monitoraggio della batteria su hardware reale:** il calcolo della percentuale da partitore resistivo è implementato, ma sul prototipo a 30 pin è disabilitato per impostazione predefinita, perché i pin ADC1 disponibili sono già occupati da ECG e termistore; la funzione resta pienamente dimostrabile tramite il simulatore, che ne alimenta anche il relativo allarme.

Questo stato riflette una precisa scelta di priorità: consolidare prima il percorso principale del dato — acquisizione, trasmissione, archiviazione, visualizzazione e allarme — e lasciare come estensioni le funzionalità accessorie.

3. Ambiente e strumenti

3.1 Stack tecnologico

Il sistema integra tecnologie diverse, ciascuna scelta per il proprio ruolo: MicroPython per il firmware del dispositivo ESP32; Python con FastAPI per il backend (API, autenticazione, canali realtime); Mosquitto come broker MQTT; InfluxDB come database di serie temporali; Node-RED come motore di flussi per l'instradamento e il supporto agli alert; Grafana per la dashboard del medico; React Native con Expo per l'app mobile; SQLite come database relazionale applicativo; Docker e Docker Compose per orchestrare l'intera infrastruttura server in modo riproducibile.

3.2 Architettura a container e porte

L'infrastruttura server è descritta in un file `docker-compose` che avvia ciascun servizio in un container isolato, permettendo di portare in esecuzione il sistema con un solo comando.

Servizio	Ruolo	Porta
Mosquitto	Broker MQTT	1883 / 9001
Node-RED	Flussi e regole	1880
InfluxDB	Database storico	8086
Grafana	Dashboard medico	3000
Backend FastAPI	API, ingest, realtime	8000

Tabella 4 — Servizi e porte dello stack Docker.

3.3 Struttura del repository

Il repository è organizzato come monorepo a moduli:

firmware/ ESP32 (MicroPython): sensori, MQTT/BLE, alert
backend/app/ FastAPI: API, auth, ingest, realtime, DB
docker-stack/ Mosquitto, InfluxDB, Node-RED, Grafana, compose
mobile/ App React Native (Expo)
scripts/ utilità e strumenti di supporto (simulatore HIL da PC)
docs/ documentazione (requisiti, architettura, sicurezza)

3.4 Viste architeturali (modello 4+1)

L'architettura può essere riletta secondo il modello 4+1. La vista logica individua le entità principali (account, dispositivo, lettura, allarme, scheda paziente). La vista di processo evidenzia i task concorrenti: sull'edge l'acquisizione del segnale, che non deve mai bloccare il resto del firmware; sul server il listener MQTT e gli endpoint REST/WebSocket, gestiti in modo asincrono. La vista di sviluppo corrisponde all'organizzazione a moduli del repository, mentre la vista fisica colloca il dispositivo alla caviglia, collegato via Wi-Fi all'host che esegue i container, a sua volta raggiunto da smartphone e browser. I casi d'uso descritti nella Sezione 2 costituiscono la vista “+1” degli scenari, che lega insieme le altre quattro.

4. Dispositivo indossabile (firmware)

4.1 Sensoristica

Il cuore del dispositivo è un microcontrollore ESP32 con firmware in MicroPython, racchiuso in una cavigliera. L'unico sensore biomedicale è un front-end ECG (AD8232), affiancato dai segnali di leads-off che rilevano quando gli elettrodi non sono a contatto con la pelle. Completano la dotazione un termistore NTC per la temperatura cutanea e, opzionalmente, un partitore di tensione per la lettura della batteria. I parametri respiratori, utili al monitoraggio di eventuali condizioni di tachipnea, vengono derivati dal segnale elettrocardiografico prelevato dall'AD8232 attraverso la tecnica EDR. È inoltre disponibile un simulatore (HIL) che produce lo stesso payload dell'hardware reale, rendendo le due modalità intercambiabili a valle.

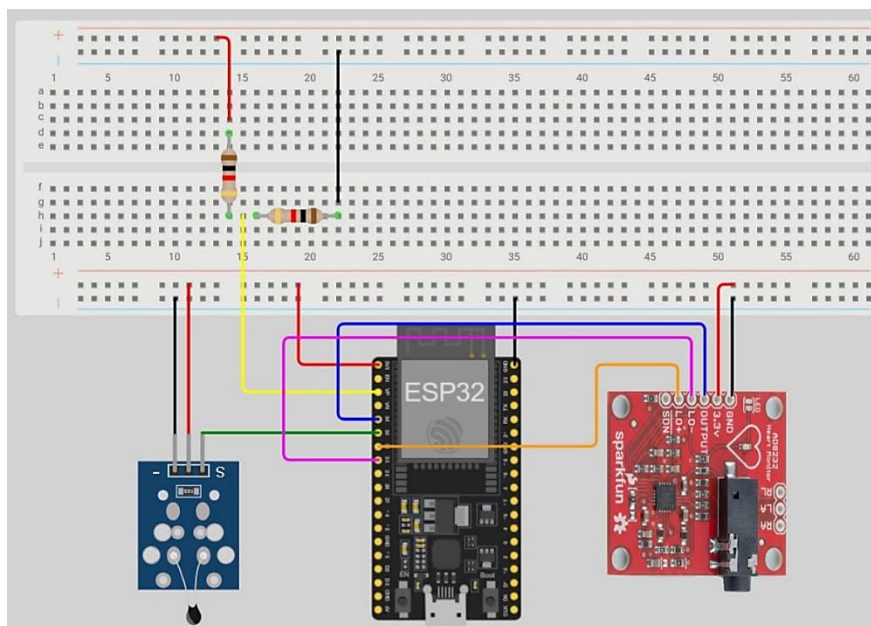


Figura 3 — Simulazione cablaggio hardware

4.2 Elaborazione a bordo

Dal segnale ECG il firmware ricava due parametri clinici, riducendo i dati grezzi a informazioni utili prima di trasmetterli (edge computing). La frequenza cardiaca è calcolata con un rilevatore di complessi QRS ispirato all'algoritmo di Pan–Tompkins — derivazione, elevamento al quadrato, soglia adattiva e periodo refrattario — da cui si misurano gli intervalli tra i picchi R. La frequenza respiratoria è invece stimata con la tecnica EDR (ECG-Derived Respiration): la respirazione modula il ritmo cardiaco (aritmia sinusale respiratoria) e questa modulazione, estratta dalla serie degli intervalli RR con un filtro IIR passa-basso e un conteggio degli attraversamenti della media, permette di stimare gli atti respiratori al minuto.

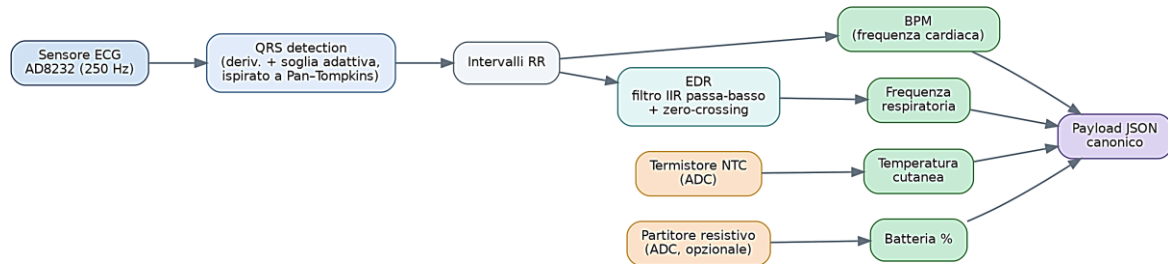


Figura 4 — Pipeline di acquisizione a bordo: dai sensori grezzi alle metriche sintetiche, fino al payload JSON canonico.

L'elaborazione a bordo ha un duplice vantaggio: riduce la quantità di dati trasmessi, perché si inviano poche grandezze sintetiche anziché il segnale grezzo ad alta frequenza, e rende il sistema più robusto, perché il calcolo dei parametri non dipende dalla rete. Il dispositivo, in altre parole, fornisce informazioni pronte all'uso. L'intero flusso è realizzato con strutture dati a dimensione fissa (ring buffer statici), per evitare allocazioni dinamiche di memoria durante l'acquisizione a 250 Hz.

4.3 Comunicazione e allarmi

Il dispositivo, identificato da un ID univoco, comunica con il backend tramite tre topic MQTT: pubblica la telemetria e gli allarmi, e resta in ascolto sul topic dei comandi per ricevere istruzioni, che applica senza riavvio. Un payload JSON unico — con identificativi, istante di misura, parametri vitali e stato del dispositivo — è condiviso da firmware, backend e app, evitando disallineamenti.

Topic	Direzione	Contenuto
.../telemetry	Device → Backend	Lecture periodiche dei parametri vitali
.../alerts	Device → Backend	Allarmi hardware/locali (batteria, guasto sensore, tachipnea locale)
.../commands	Backend → Device	Comandi (paziente, frequenza di invio)

Tabella 5 — Topic MQTT del dispositivo.

Oltre alla telemetria, il firmware genera in autonomia alcuni allarmi locali (batteria scarica, guasto del sensore di temperatura, elettrodi scollegati, tachipnea rilevata a bordo) su un topic dedicato, in modo da segnalare una condizione anomala anche in caso di problemi a valle. Per evitare falsi positivi, una condizione viene segnalata solo dopo alcune letture anomale consecutive (debounce), e alla risoluzione viene inviato un messaggio informativo di rientro.

Lo stato complessivo del dispositivo è riassunto in un codice testuale (`device_status`) che il firmware calcola a ogni ciclo di invio, come mostrato dalla macchina a stati seguente.

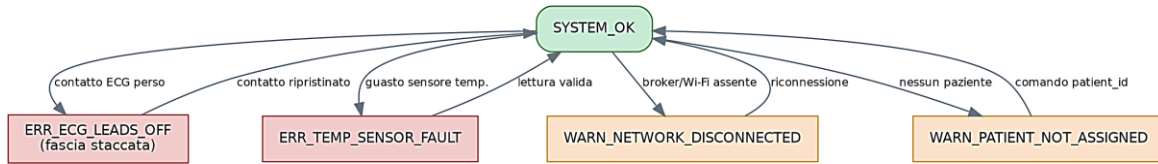


Figura 4 — Stati diagnostici del dispositivo (*device_status*) e transizioni principali.

5. Backend e gestione dei dati

5.1 API REST e autenticazione

Il backend, sviluppato con FastAPI, espone le API per registrazione, login, lettura dell'ultima telemetria, storico delle letture e degli allarmi, statistiche aggregate, gestione della scheda paziente, invio comandi, configurazione delle soglie e consultazione dell'audit log. L'autenticazione è basata su token JWT (JSON Web Token), con password protette da hashing bcrypt, e distingue due ruoli (caregiver e medico). Una dipendenza di autorizzazione garantisce che ogni utente acceda solo ai dispositivi di propria competenza, assicurando l'isolamento dei dati.

Endpoint	Scopo
/register, /login	Registrazione e autenticazione (token, ruolo, device)
/devices/{id}/latest history alerts	Ultima lettura, storico letture e allarmi
/devices/{id}/stats	Statistiche aggregate (media, min, max)
/devices/{id}/patient	Lettura/aggiornamento della scheda paziente
/devices/{id}/thresholds	Configurazione soglie cliniche (solo medico in scrittura)
/devices/{id}/commands	Invio comandi al dispositivo
/ws/live, /sse/live	Canali realtime (WebSocket e SSE)
/audit	Consultazione audit log (solo medico)

Tabella 6 — Principali gruppi di endpoint del backend.

5.2 Ingest e canali in tempo reale

All'avvio, il backend attiva un processo in background sottoscritto ai topic della telemetria e degli allarmi: ogni messaggio viene validato, registrato e, se necessario, valutato rispetto alle soglie. Lo stesso dato viene poi ritrasmesso a tutti i client connessi via WebSocket, così che l'app si aggiorni in tempo reale senza interrogare ripetutamente il server. Il backend espone anche un canale Server-Sent Events (SSE) equivalente, come alternativa al WebSocket per client che non lo supportano (ad esempio alcuni browser); l'app mobile utilizza il canale WebSocket.

5.3 Persistenza: database relazionale e InfluxDB

La persistenza segue i due percorsi descritti nell'architettura: il backend utilizza un database relazionale (SQLite) per le entità applicative (account, dispositivi, letture, allarmi, schede paziente, soglie e audit log) su cui poggiano l'app e la logica di autorizzazione. In parallelo, le serie temporali cliniche confluiscono in InfluxDB, alimentato da Node-RED, che è la sorgente dei grafici storici di Grafana. La scrittura su InfluxDB direttamente dal backend è prevista come opzione ma disabilitata per impostazione predefinita, per non duplicare i punti dati rispetto al percorso Node-RED. I due piani sono complementari: il primo è orientato alle relazioni e ai permessi, il secondo all'analisi temporale dei segnali.

5.4 Soglie cliniche e gestione degli alert

Il backend valuta le letture rispetto a soglie cliniche e genera allarmi distinti per gravità (attenzione, critico, tecnico). Ogni allarme riporta il dispositivo, il parametro, la descrizione, la gravità e l'istante. Le condizioni rilevate riguardano la frequenza respiratoria (tachipnea/bradipnea), la frequenza cardiaca (tachicardia/bradicardia) e la temperatura (febbre/ipotermia). Vale una regola “antipanico”: quando il sensore non è a contatto si emette solo un allarme tecnico, mai allarmi fisiologici. Le soglie sono configurabili per singolo dispositivo dal medico, con un controllo di coerenza tra i valori; in assenza di configurazione si usano valori di default.

Nota. La valutazione clinica completa sui tre parametri risiede nel backend. Il firmware, invece, genera in autonomia solo l'allarme clinico di tachipnea (oltre agli allarmi hardware): questa ripartizione mantiene il dispositivo leggero e concentra la logica configurabile lato server.

5.5 Scheda paziente e auto-assegnazione

La scheda paziente raccoglie l'anagrafica e le informazioni cliniche essenziali (patologie note, farmaci, allergie), accessibili al proprietario e al medico. L'associazione paziente–dispositivo è automatica: alla registrazione il backend genera l'identificativo paziente, associa il dispositivo all'account, crea la scheda e notifica il firmware via comando MQTT (con invio best-effort), così che la telemetria sia subito intestata al bambino senza passaggi manuali.

6. Modello dei dati

La persistenza relazionale del backend è organizzata attorno a poche entità chiare, allineate al modello entità–relazione del progetto. La tabella ne riassume il ruolo, e il diagramma successivo ne mostra attributi e cardinalità.

Entità	Ruolo
Caregiver	Account utente, con ruolo caregiver o medico
Device	Dispositivo fisico, associato a un caregiver
Reading	Singola lettura dei parametri vitali nel tempo
Alert	Allarme generato (parametro, gravità, messaggio, istante)
DeviceThreshold	Soglie cliniche per dispositivo, impostate dal medico
PatientRecord	Scheda del paziente: anagrafica e anamnesi
AuditLog	Registro append-only delle operazioni rilevanti
PushToken	Token per le notifiche push del caregiver

Tabella 7 — Entità principali del modello dati relazionale.

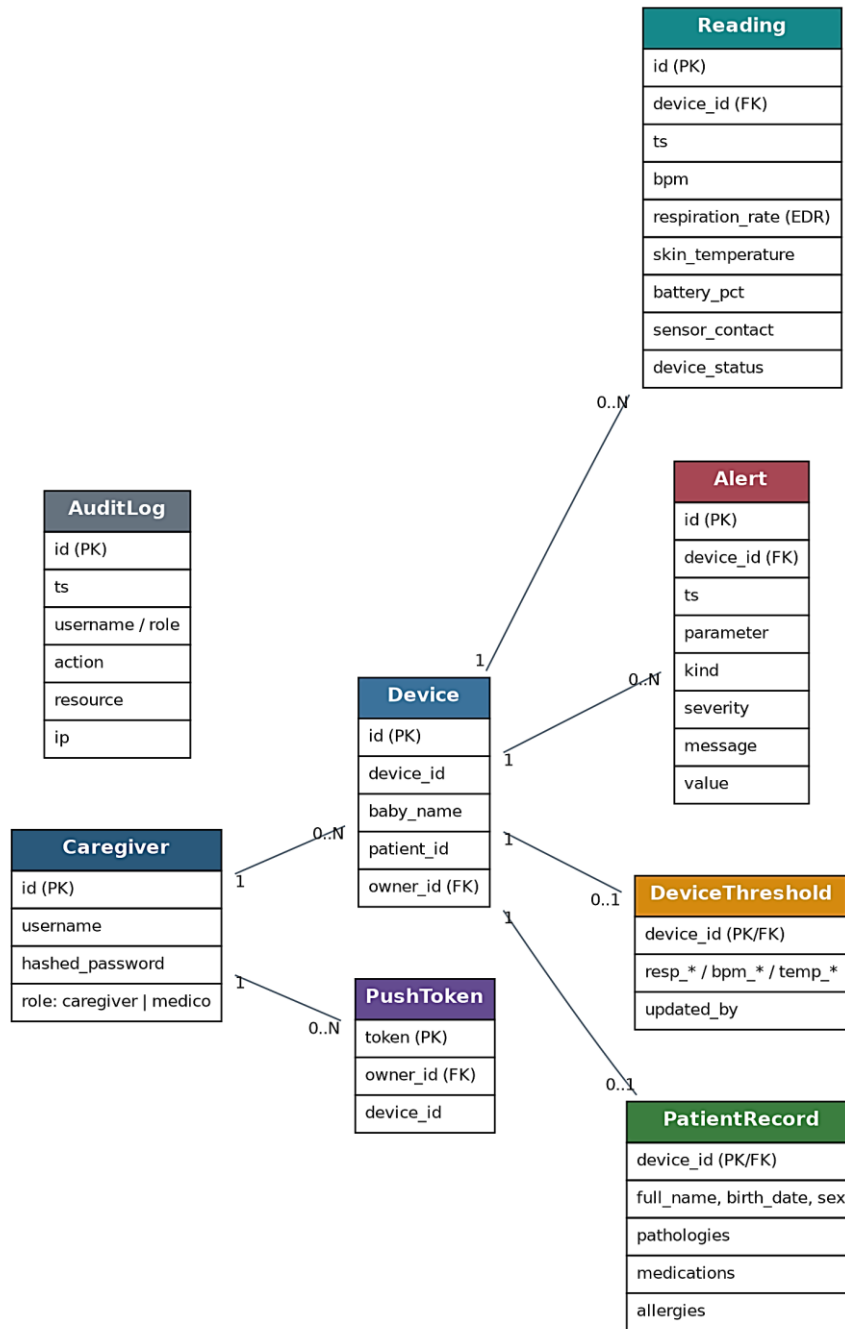


Figura 6 — Modello dei dati relazionale (diagramma E-R / classi) con attributi e cardinalità.

Le cardinalità riflettono il dominio: un caregiver possiede zero o più dispositivi; ogni dispositivo genera molte letture e molti allarmi e ha al più una scheda paziente e un insieme di soglie. Un dettaglio progettuale utile è che un dispositivo può esistere anche senza proprietario: la telemetria può arrivare prima che l'account venga registrato, e l'associazione successiva “intesta” retroattivamente i dati al paziente.

Come già osservato, le letture vivono su due piani complementari: nel database relazionale, dove alimentano l'app, lo storico e gli allarmi con le rispettive relazioni e permessi; e, in parallelo, su InfluxDB, dove sono ottimizzate per l'analisi temporale dei grafici di Grafana. Questa duplicazione è voluta e rispecchia i due percorsi paralleli dell'architettura.

7. Protocolli di comunicazione

Questa sezione riassume, a livello logico, i due dialoghi principali del sistema, documentati in dettaglio nella cartella docs del repository.

7.1 Firmware ↔ Backend

All'accensione il dispositivo si connette al Wi-Fi e sincronizza l'orario (tramite il protocollo NTP), in modo da marcare le letture con un timestamp coerente. Stabilisce poi la connessione al broker MQTT e inizia a pubblicare la telemetria a cadenza regolare. In parallelo resta sottoscritto al topic dei comandi: quando il backend invia una configurazione (ad esempio una nuova frequenza di invio o l'associazione di un paziente), il firmware la applica. Gli eventuali allarmi hardware viaggiano su un terzo topic dedicato; una macchina a stati cooperativa gestisce le riconnessioni in caso di caduta del Wi-Fi o del broker, senza interrompere l'acquisizione.

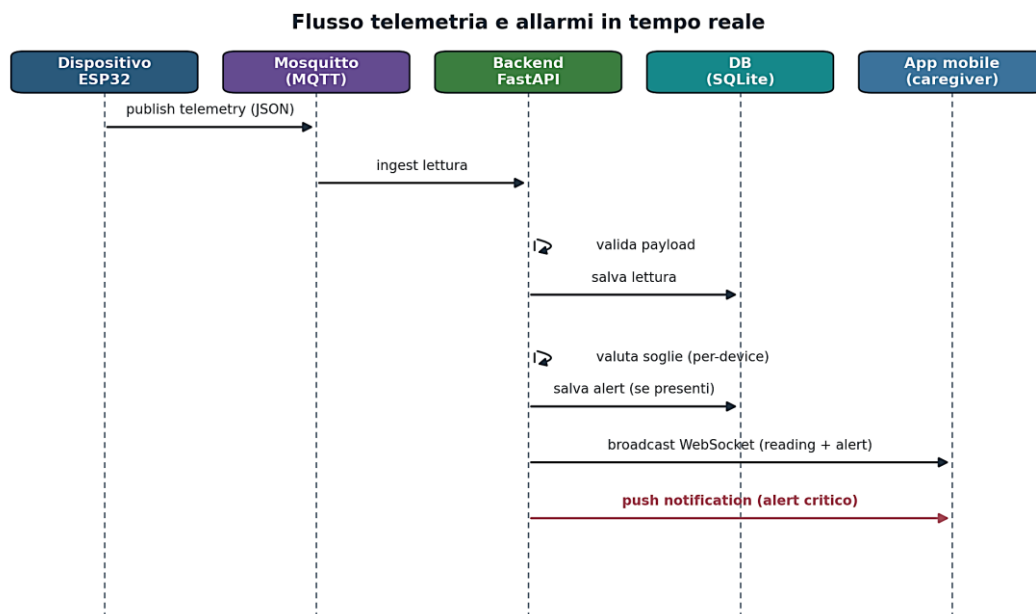


Figura 7 — Diagramma di sequenza (UML): dalla pubblicazione della telemetria all'aggiornamento in tempo reale dell'app, con notifica push sull'allarme critico.

7.2 App ↔ Backend

L'app inizia con la registrazione o il login: in caso di successo riceve un token JWT, il ruolo e l'identificativo del dispositivo, conservati in storage cifrato. Da quel momento ogni richiesta REST è autenticata con il token. Per i dati in tempo reale, l'app apre un canale WebSocket (passando il token) e riceve gli aggiornamenti appena arrivano; in caso di disconnessione si riconnette automaticamente con attese crescenti e, nel frattempo, recupera l'ultimo valore via REST come rete di sicurezza. Lo storico, gli allarmi e l'invio dei comandi passano per le API REST; le notifiche push, dove configurate, raggiungono il caregiver anche ad app chiusa.

7.3 Configurazione da remoto

Un terzo scenario tipico è la modifica di un parametro del dispositivo. Dall'app, il caregiver invia un comando — ad esempio una nuova frequenza di invio — al backend, che verifica i permessi e lo inoltra sul topic MQTT dei comandi: il dispositivo, sottoscritto a quel topic, riceve il messaggio, aggiorna la propria configurazione e riprende a pubblicare la telemetria alla nuova cadenza, senza bisogno di riavvio.

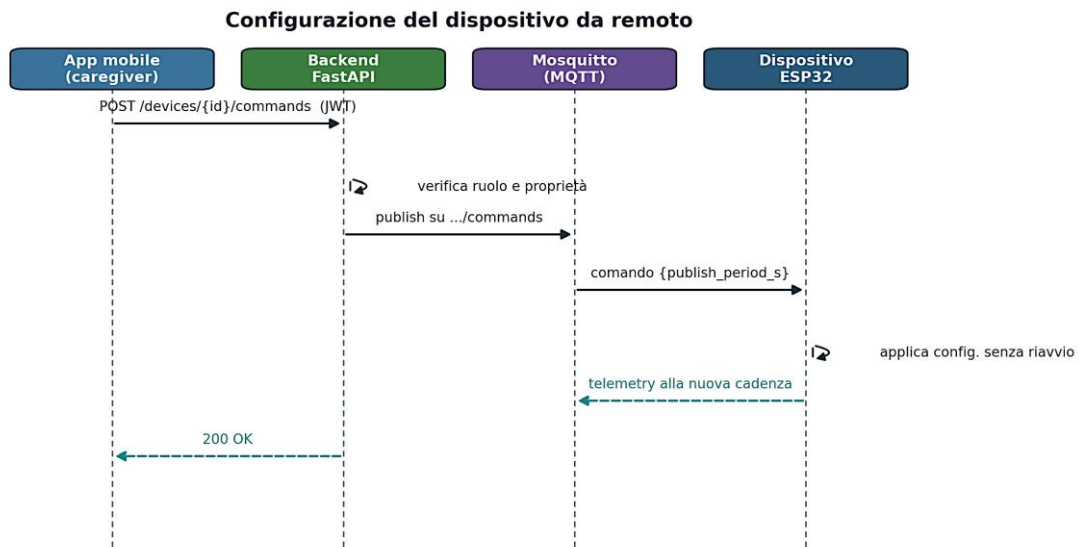


Figura 8 — Diagramma di sequenza (UML) della configurazione da remoto: il comando viaggia *app* → *backend* → *MQTT* → *dispositivo*.

8. Applicazione mobile

8.1 Architettura a livelli

L'app, in React Native con Expo, è organizzata a livelli con responsabilità separate: configurazione, accesso ai dati (chiamate REST con time-out esplicito e canale WebSocket), notifiche e schermate di presentazione. Una modalità dimostrativa (DEMO_MODE), attiva per impostazione predefinita, consente di mostrare l'intera interfaccia con dati simulati mantenendo le stesse firme delle funzioni reali; questo permette di sostituire lo strato dati senza toccare le schermate e di provare l'app anche senza backend attivo.

8.2 Schermata di monitoraggio e resilienza

La schermata principale mostra in tempo reale le tre misurazioni (frequenza cardiaca, frequenza respiratoria, temperatura), lo stato del dispositivo e della batteria, i banner di allarme attivi, l'elenco degli allarmi recenti e lo storico delle letture; la parte alta riporta lo stato di connessione e i dati del paziente. I dati arrivano via WebSocket: in caso di disconnessione, l'app si riconnette con attese crescenti (backoff esponenziale) e, nel frattempo, recupera l'ultimo valore via polling REST, evitando schermate vuote o bloccate.

L'interfaccia è pensata per un genitore, non per un tecnico: i valori sono grandi e leggibili, i colori comunicano immediatamente la normalità o l'anomalia, e gli allarmi più gravi sono evidenziati con banner a piena larghezza. L'obiettivo è che basti un'occhiata per capire se tutto procede regolarmente.

8.3 Notifiche e sicurezza della sessione

Gli allarmi vengono mostrati come notifica locale ad app aperta e, dove configurato, come notifica push (tramite il servizio Expo) anche ad app chiusa, con titolo e priorità variabili in base alla gravità. Il token di sessione e l'identificativo del dispositivo sono conservati in storage cifrato (SecureStore) e mai in chiaro, evitando di richiedere un nuovo accesso a ogni apertura.

9. Dashboard medico (Grafana)

La dashboard Grafana è la vista riservata al medico, alimentata dai dati archiviati in InfluxDB e provisionata automaticamente all'avvio dello stack. Permette di sorvegliare i pazienti a colpo d'occhio: dati correnti, grafici storici di ciascun parametro, filtri per paziente e dispositivo, un indicatore di stato complessivo derivato dagli allarmi attivi, il conteggio degli allarmi e una scheda con le informazioni cliniche essenziali. Essendo un'applicazione web autonoma, gestisce in proprio l'interattività (selezione dell'intervallo temporale, zoom sui grafici).

10. Sicurezza e privacy

La sicurezza è affrontata su più livelli, coerenti con la natura clinica dei dati trattati, ispirandosi ai principi del GDPR (Regolamento Generale sulla Protezione dei Dati): minimizzazione dei dati, controllo degli accessi e tracciabilità delle operazioni. Trattandosi di un prototipo didattico, alcune misure tipiche di un contesto di produzione — in primo luogo la cifratura dei canali di trasporto — restano come evoluzione naturale (cfr. §10.3 e §13.3).

10.1 Autenticazione e ruoli

L'accesso ai dati richiede sempre autenticazione: il login restituisce un token JWT con una scadenza esplicita (campo exp) che ne limita la validità temporale, insieme al ruolo dell'utente; le password sono protette con hashing bcrypt e mai memorizzate in chiaro. Sono previsti due ruoli, con permessi differenziati.

Ruolo	Permessi
Caregiver (paziente)	Vede e gestisce solo i propri dispositivi e la relativa scheda paziente
Medico	Vede tutti i pazienti, configura le soglie cliniche e consulta l'audit log

Tabella 8 — Ruoli e permessi (RBAC).

10.2 Isolamento dei dati e audit log

Ogni accesso ai dati di un dispositivo passa da un controllo di proprietà: un caregiver può accedere solo ai dispositivi di cui è proprietario, mentre il medico ha visibilità completa. Questo realizza il principio per cui ogni utente vede esclusivamente i propri dati. Tutte le operazioni rilevanti — login, registrazioni, associazione di un dispositivo, lettura di storico e allarmi, modifica delle soglie, invio comandi — sono registrate in un audit log append-only che traccia chi, cosa, su quale dispositivo, quando e da quale indirizzo; il registro è consultabile solo dal medico.

10.3 Gestione dei segreti e avvertenze

Le credenziali Wi-Fi/MQTT del dispositivo e le variabili d'ambiente del server sono mantenute fuori dal repository, tramite file locali esclusi dal versionamento e basati su template con soli segnaposto; tutto il traffico clinico resta sulla rete locale. Trattandosi di un progetto didattico, restano come naturale evoluzione la cifratura dei canali di trasporto (TLS/HTTPS) e, in un contesto reale, la creazione degli account medico da parte di un amministratore anziché per auto-registrazione.

11. Scelte progettuali di sintesi

A chiusura della descrizione tecnica, le decisioni che hanno maggiormente caratterizzato la progettazione, con la relativa motivazione, sono riassunte nella tabella di seguito.

Scelta	Motivazione
Un solo sensore ECG (BPM + respiro via EDR)	Meno ingombro e meno punti di guasto su un indossabile pediatrico; il respiro è il segnale più utile per l'asma
Elaborazione a bordo (edge computing)	Riduce i dati trasmessi e rende il calcolo indipendente dalla rete
Due percorsi di persistenza paralleli	Database relazionale (SQLite) per app e permessi; InfluxDB per i grafici storici
Regola antipanico sugli allarmi	Con sensore staccato solo un avviso tecnico, mai falsi allarmi fisiologici
Stesso payload per simulatore e hardware	Passaggio trasparente tra demo e dispositivo reale
Infrastruttura containerizzata	Ambiente riproducibile e avviabile con un solo comando

Tabella 9 — Principali scelte progettuali e motivazioni.

Il filo conduttore è l'equilibrio tra semplicità, robustezza e fedeltà clinica: ogni scelta riduce la complessità, dove possibile, senza rinunciare all'informazione realmente utile per il monitoraggio dell'asma.

12. Test e validazione

Il sistema è stato verificato end-to-end, controllando che ogni anello della catena funzionasse insieme agli altri:

- **Acquisizione e pubblicazione:** il dispositivo (o il simulatore) acquisisce i segnali, deriva BPM e frequenza respiratoria e pubblica la telemetria.
- **Ingest e archiviazione:** il backend riceve, valida e storicizza le letture, che risultano interrogabili per ultimo valore e per storico.
- **Realtime:** l'app riceve gli aggiornamenti via WebSocket e ricade correttamente sul polling REST in caso di disconnessione.
- **Propagazione degli allarmi:** una condizione di tachipnea genera l'allarme, lo fa comparire nell'app come banner e notifica e lo rende visibile nella dashboard.
- **Controllo:** la registrazione associa automaticamente il paziente; un comando di configurazione viene recepito dal firmware senza riavvio.

Le verifiche confermano che la pipeline è coerente e che i parametri visualizzati corrispondono, campo per campo, a quanto effettivamente prodotto dal dispositivo.

13. Conclusioni

13.1 Obiettivi raggiunti

Il team ha realizzato l'intera catena IoT di Alvea: un dispositivo indossabile che deriva da un solo ECG il battito e la frequenza respiratoria; un backend con autenticazione a ruoli, ingest MQTT, persistenza e realtime; un'app mobile per il genitore e una dashboard Grafana per il medico; e una gestione degli allarmi propagati fino all'app, con notifiche. I requisiti funzionali e non funzionali prioritari risultano soddisfatti.

13.2 Valutazione del risultato e limiti

Il risultato è un sistema completo e coerente, che copre l'intero percorso del dato dal sensore al medico. Sul piano clinico, il parametro più specifico per l'asma è la frequenza respiratoria, ricavata via EDR, utilizzata per il

monitoraggio di un possibile stato di tachipnea (segno precoce di insufficienza respiratoria); il battito è di supporto e la temperatura ha valore informativo. La qualità della stima respiratoria dipende dalla bontà del segnale ECG e può degradarsi in presenza di artefatti da movimento, tipici di un bambino. Il sistema, inoltre, non misura la saturazione di ossigeno né il picco di flusso espiratorio. Va ribadita la natura didattica del progetto, non destinato a un uso clinico reale.

Dal punto di vista ingegneristico, il valore del progetto sta nell'integrazione: non un singolo componente isolato, ma una catena completa in cui sensore, comunicazione, archiviazione, visualizzazione e sicurezza cooperano secondo contratti ben definiti. È questa coerenza end-to-end a rendere il sistema dimostrabile e, soprattutto, estendibile: ogni miglioramento futuro può innestarsi su un'infrastruttura già funzionante.

13.3 Sviluppi futuri

Tra i possibili sviluppi: la cifratura dei canali (TLS/HTTPS); l'aggancio pieno del ruolo medico/paziente in ogni schermata dell'app; l'abilitazione di un consumatore BLE lato app; un meccanismo di rilevamento dell'assenza di dati che segnali quando il dispositivo è offline da troppo tempo; l'analisi di tendenza dei parametri nel tempo per anticipare la segnalazione di un peggioramento; e una gestione più robusta degli account medico. Nel complesso, Alvea costituisce una solida base per il monitoraggio domiciliare dell'asma pediatrico.

Appendice A — Glossario e acronimi

Termine	Significato
IoT	Internet of Things: dispositivi connessi in rete
ECG	Elettrocardiogramma: segnale elettrico dell'attività cardiaca
EDR	ECG-Derived Respiration: stima del respiro dal segnale ECG
BPM	Battiti per minuto (frequenza cardiaca)
MQTT	Protocollo di messaggistica publish/subscribe per l'IoT
JWT	JSON Web Token: token firmato per l'autenticazione
RBAC	Role-Based Access Control: controllo degli accessi per ruolo
WebSocket / SSE	Canali per l'invio di dati in tempo reale al client
NTC	Termistore a coefficiente di temperatura negativo
BLE	Bluetooth Low Energy: trasmissione radio a basso consumo
HIL	Hardware-in-the-Loop: simulatore che emula il dispositivo reale
Tachipnea	Aumento anomalo della frequenza respiratoria

Tabella 10 — Glossario dei principali termini e acronimi.